Colony — A Playthrough

Transcript of the first Colony game. Every move is a substrate primitive.

Setup

```
        ·  ·  ·  ·  ·  ·  ·
     ·  ·  ·  🍯  ·  ·  ·
  ·  ·  ·  ·  ·  ·  🍯  ·
·  ·  🍯  ·  [N]  ·  ·  ·
  ·  ·  ·  ·  ·  🍯  ·  ·
     ·  ·  ·  ·  ·  ·  ·
        ·  ·  🍯  ·  ·  ·
```

Starting colony: 1 queen, 2 scouts, 50 food. No trails. Fog everywhere.

Turn 1 — Hatch Scout

Action: `hatch scout` (-10 food)

Three scouts scatter outward. Balanced mood (0.5 sensitivity) means all paths look roughly equal. No trails to follow — pure exploration.

```
        ·  ·  ·  ·  ·  ·  ·
     ·  ·  ·  🍯  ·  ·  ·
  ·  ·  ·  🐜→ ·  ·  🍯  ·
·  ·  🍯  ·  [N]  ·  ·  ·
  ·  ·  ·  ←🐜  ·  🍯  ·  ·
     ·  ·  ·  ·  🐜→ ·  ·
        ·  ·  🍯  ·  ·  ·
```

- Scout A headed NW — nothing yet, fog ahead
- Scout B headed SW — moving toward food
- Scout C headed SE — fog ahead

```
Food: 40 | Ants: 4 | Trails: none
```

Substrate: `nest.hatch('scout')` → `add()` a new unit.

Turn 2 — Wait

Scouts still moving. Nothing to do but watch. Scout B is one turn from food SW.

```
⚡ fade() — nothing to decay yet
```

Turn 3 — Scent SW (Scout B Finds Food)

Action: `scent SW` — drop pheromone toward food, reinforcing Scout B's path.

Scout B found food! 🍯

She deposits pheromone on the return trail. The colony's first path is born.

```
        ·  ·  ·  ·  ·  ·  ·
      ·  ·  🐜A  🍯  ·  ·  ·
    ·  ·  ·  ·  ·  ·  🍯  ·
  ·  ·  🍯  ·  [N]  ·  ·  ·
    ·  ·  · /·  ·  🍯  ·  ·
      ·  · / ·  ·  ·  🐜C  ·
        ·  🍯← 🐜B  ·  ·  ·
```

```
Trail: N→SW  ▒▒·····  str 1 | res 0  (fresh, fragile)
Food: 40 | Ants: 4
```

Substrate: `enqueue({ receiver: 'scout:explore', data: { hex: SW } })` → scout finds food → `deposit('N→SW', 1)` → first `mark()`.

Turn 4 — Hatch Harvester, Scout B Returns

Action: `hatch harvester` (-20 food)

Scout B returns to the nest, depositing pheromone the whole way. Trail doubles in strength. The harvester smells the fresh trail immediately — sensitivity 0.9, she locks on.

```
        ·  ·  ·  ·  ·  ·  ·
      ·  ·  🐜A  🍯  ·  ·  ·
    ·  ·  ·  ·  ·  ·  🍯  ·
  ·  ·  🍯  ·  [N]🐜B ·  ·  ·
    ·  ·  · /·  ·  🍯  ·  ·
```

```
      ·  · / ·  ·  ·  🐜C  ·
        ·  🍯  ·  ·  ·  ·  ·
```

🐝 Harvester → following N→SW highway

```
Trail: N→SW  ▒▒▒▒▒▒▒▒·····  str 2 | res 0
Food: 20 | Ants: 5
```

The first feedback loop forms:

```
harvester follows trail → reaches food → carries back → deposits pheromone
         ↑                                                          │
         └──────────── trail gets stronger ← ──────────────────────┘
```

Substrate: `hatch('harvester')` → `add()` unit with sensitivity 0.9 → `follow()` strongest path.

Turn 5 — Scent NW (Scout A Finds Food)

Action: `scent NW` — push Scout A toward the NW food source.

Food found NW! 🍯

Second trail born. Meanwhile the harvester reaches food SW, loads up.

```
        ·  ·  ·  ·  ·  ·  ·
      ·  ·  🍯←🐜A  ·  ·  ·
    ·  ·  · \ ·  ·  ·  🍯  ·
  ·  ·  🍯  · \[N]  ·  ·  ·
    ·  ·  · /·  ·  🍯  ·  ·
      ·  · / ·  ·  ·  ·  ·
        ·  🍯  🐝  ·  ·  ·  ·
```

```
Trails: SW str 1.8 | NW str 1 (new)
Food: 15 | Ants: 5
```

Two trails. One harvester. The NW trail has no harvester — it'll fade.

The tension: hatch another harvester (costs 20, have 15), or wait for delivery?

Turns 6–14 — The Colony Grows

Ten turns of focused play. The strategy: **explore early, exploit hard, defend late.**

Turn 6 — Harvester Delivers. Hatch Harvester #2.

🐝 arrives with +15 food. Deposits pheromone. SW trail hits str 2.5.
Immediately hatch harvester #2 (-20). Send her NW.
Scout C breaks through SE fog — **finds a double food source** 🍯🍯 .

```
Food: 10 | Ants: 6 | Trails: SW 2.5, NW 1.2, SE 0.5 (new)
```

Turn 7 — Scent SE. Mood → Exploit (0.7)

Third trail is rich but fragile. Drop scent so Scout C reinforces it.
Shift mood to 0.7 — harvesters lock harder onto highways.
Both harvesters cycling. Food starts flowing.

```
Food: 18 | +13 delivered, -5 upkeep
```

Substrate: Colony-wide sensitivity shift. `weight = 1 + trail × 0.7` — harvesters discriminate more strongly between trails.

Turn 8 — Hatch Harvester #3 for SE

Economy is positive. Spend 20 on a third harvester for the double source.
Scout A redeployed north into deep fog.

```
Food: 11 | Ants: 7 | Three harvesters on three trails
```

Turn 9 — The Machine Kicks In

All three harvesters cycling. SW and NW delivering every turn.
SE harvester reaches double source — **loads +20**.

```
Food: 34 | +28 delivered, -7 upkeep | Economy: POSITIVE
```

This is emergence. Nobody coded "efficient foraging." The pheromone math did it: shorter paths get more round-trips, more round-trips deposit more pheromone, more pheromone attracts more ants. Shortest path wins automatically.

Turn 10 — Hatch 2 Scouts. Mood → 0.6

Surplus lets you explore again. Two fresh scouts hatch (-20).
Drop mood slightly to balance. Five scouts scatter into fog.
SW trail now str 4.2 — approaching highway.

```
Food: 41 | Ants: 9 | SW 4.2, NW 2.8, SE 3.1
```

Turn 11 — Contact

Scout A: food N 🍯 . New trail seeded.
Scout D: **enemy ant spotted NE** ⚠️ . They're scouting toward your NW food.

```
Food: 52 | Trails: 4 | ⚠️  FIRST CONTACT
```

Turn 12 — Hatch Guard. Alarm NE.

First guard (-25). Drop alarm pheromone on the NE approach.
Guard patrols the corridor between NW food and enemy approach.

```
Food: 34 | Ants: 10 | Guard patrolling NE
```

Substrate: `warn(NE_corridor, 1)` — resistance building on enemy path. Their `weight = 1 + (strength − resistance) × sensitivity` drops. If resistance > 2× strength, the path goes **toxic** — enemy ants won't use it.

Turn 13 — Hatch Harvester #4. First Highway.

Four harvesters, four trails. SE double source is a machine.
SW trail hits str 5.0 — **HIGHWAY** 🛤️
Guard deposits alarm on two enemy hexes. Their scout turns back.

```
Food: 49 | SW: ██████████ HIGHWAY | NE: res 2.3 (walled)
```

Turn 14 — Imprint. Mood → Exploit (0.8)

Action: `imprint SW` — permanently lock the strongest trail.

`nest.imprint()` — **knowledge +1**. This trail will never fade.

Push mood to 0.8. Harvesters swarm the proven paths.

Substrate: `know()` — promote highway to permanent knowledge in TypeDB. Loop L6 fires.

Turn 15 — The Colony

```
        ·  ·  ·  🍯  ·  ·  ·
     ·  ·  🍯←🐝·  ·  ·  ·
  ·  ·  ·  ╲🐜·  ⚠️·  🍯  ·
```

```
  ·  ·  🍯   · \[N]/ 🌹   ·  ·
    ·  ·  · /·  /·  🍯   ·  ·
      ·  · / · /·  ·  ·  ·
        ·  🍯 🐝🐝  ·  ·  ·
                ↓
            🍯🍯 (double)
```

```
┌──── COLONY STATE ──────────────────────────┐
│  Food:  56          Ants:  11              │
│  Queen: 1   Scouts: 5   Harvesters: 4 │
│  Guards: 1  Nurses: 0                      │
│                                            │
│  Trails:                                   │
│    N→SW  ▓▓▓▓▓▓▓▓▓▓  str 5.0  IMPRINTED│
│    N→NW  ▓▓▓▓▓▓▒▒▒▒  str 3.4 | res 0      │
│    N→SE  ▓▓▓▓▓▓▓▒▒▒  str 4.1 | res 0      │
│    N→N   ▓▓▓▒▒▒▒▒▒▒  str 1.8 | res 0      │
│                                            │
│  Defense:                                  │
│    NE    ⚠️ alarm    str 0   | res 2.3  │
│                                            │
│  Mood:  ◄──────●──► (0.8 exploit)     │
│  Knowledge: 1 (SW imprinted)               │
│                                            │
│  Economy: +21/turn after upkeep            │
│  Enemy: 1 scout spotted, turned back       │
└────────────────────────────────────────────┘
```

What Happened (The Substrate View)

Fifteen turns. Every decision was a substrate primitive:

Turn	Player Action	Substrate Call
1	Hatch scout	`add('scout')`
2	Wait	`fade()`
3	Scent SW	`enqueue()` → scout finds food → `mark('N→SW', 1)`
4	Hatch harvester	`add('harvester')` → `follow()` strongest path
5	Scent NW	`enqueue()` → scout finds food → `mark('N→NW', 1)`
6	Hatch harvester #2	`add()` → economy starts
7	Mood → 0.7	Colony sensitivity shift
8	Hatch harvester #3	`add()` → three trails working

Turn	Player Action	Substrate Call
9	Wait	Harvesters cycle → `mark()` accumulates
10	Hatch 2 scouts	`add()` × 2 → explore with surplus
11	—	Scout reports: food N, enemy NE
12	Hatch guard, alarm NE	`add('guard')` → `warn('NE', 1)`
13	Hatch harvester #4	`add()` → SW reaches highway
14	Imprint SW	`know()` → permanent trail
15	—	Colony state: +21/turn, 4 trails, 1 highway

The Lessons

1. Explore/Exploit Is THE Decision

Early: scouts everywhere, mood low, find the map.
Mid: shift to exploit, harvesters on every trail, economy roars.
Late: knowledge locks in what works. One slider. Deepest decision in the game.

2. Emergence Is Real

Nobody coded "find shortest path." Shorter paths get more round-trips.
More round-trips deposit more pheromone. More pheromone attracts more ants.
The optimal foraging pattern falls out of the math.

3. Asymmetric Decay Rewards Builders

Resistance fades 2× faster than strength. The guard's alarm wall on NE is expensive to maintain. But trails compound. **Building is cheaper than fighting.**

The substrate's formula: `weight = 1 + max(0, strength − resistance) × sensitivity`

Building adds strength (persists). Fighting adds resistance (fades fast).
Over time, the builder's trails become highways. The fighter's walls crumble.

4. The Feedback Loop Is Everything

```
hatch → explore → find → harvest → deposit → trail strengthens → harvest
more
```

Once this loop starts, the colony grows exponentially. The player who closes the loop first wins — not the one who clicks fastest.

5. Knowledge Is Permanent

Trails fade. Highways fade slower. But imprinted trails **never fade**.
The `know()` call promotes a trail from runtime memory to TypeDB — from nervous system to brain. It's the only permanent action in the game.

Choose carefully what you make permanent.

What's Next

The colony has momentum. Four trails, positive economy, one enemy repelled.
The map still has fog. More food sources hide in the darkness.

Three paths forward:

Expand — Send scouts into remaining fog. Find food before the enemy does.
More trails, more harvesters, more imprints.

Fortify — Mass guards on NE. Push alarm pheromone into enemy territory.
Wall off their expansion. Starve them out.

Transcend — Imprint 10 trails for knowledge victory. Ignore the enemy.
Build highways so strong that the colony becomes unkillable.

The explore/exploit slider sits at 0.8.

The colony hums.

You don't play the ants. You play the colony. The ants play themselves.

Behind the Scenes — What TypeDB Actually Does

Every game action triggers real transactions across two layers. The runtime (nervous system) handles millisecond decisions. TypeDB (brain) persists the colony's memory. Here's what happens for every move in the playthrough.

The Two Layers

```
┌──────────────────────────────────────────────────────────────┐
│  RUNTIME (world.ts)                in-memory, per-tick       │
│                                                              │
│  strength: Record<string, number>   { "N→SW": 2.5 }         │
```

```
│  resistance: Record<string, number> { "NE": 2.3 }      │
│  peak: Record<string, number>       { "N→SW": 5.0 }    │
│  queue: Signal[]                    [{ receiver: ... }] │
│  units: Record<string, Unit>        { scout: Unit }    │
│                                                        │
├────────────────────────────────────────────────────────┤
│  TYPEDB (persist.ts)               durable, queryable  │
│                                                        │
│  unit entity         — uid, model, system-prompt, gen  │
│  path relation       — strength, resistance, revenue   │
│  signal relation     — sender, receiver, data, ts      │
│  skill entity        — skill-id, price, tags           │
│  capability relation — provider, offered, price        │
│  hypothesis entity   — statement, status, p-value      │
│  frontier entity     — type, description, expected-val │
│                                                        │
└────────────────────────────────────────────────────────┘
```

The runtime is fast and volatile. TypeDB is slow and permanent. Every `mark()`, `warn()`, and `fade()` writes to both simultaneously.

Turn 1: Hatch Scout

Runtime — `world.add('scout')`:

```
// world.ts:174 — creates unit, drains any queued signals
const u = unit('scout', (s, from) => signal(s, from))
units['scout'] = u
// drain queued signals for this unit (none yet)
```

TypeDB — `persist.actor('scout', 'agent')`:

```
insert $u isa unit,
  has uid "scout",
  has unit-kind "agent",
  has status "active",
  has success-rate 0.5,
  has activity-score 0.0,
  has sample-count 0,
  has reputation 0.0,
  has balance 0.0,
  has generation 0;
```

What's stored: A new unit entity with default stats. Generation 0 means it's never been evolved. Success rate starts at 0.5 (neutral).

Turn 3: Scout B Finds Food — First mark()

Runtime — scout signals food found, trail gets marked:

```
// world.ts:117-121 — mark strengthens path, records peak
strength['N→SW'] = (strength['N→SW'] || 0) + 1  // → 1
peak['N→SW'] = Math.max(peak['N→SW'] || 0, 1)   // → 1
lastUsed['N→SW'] = Date.now()
```

TypeDB — `persist.mark('N→SW', 1)`:

```
match
  $from isa unit, has uid "N";
  $to isa unit, has uid "SW";
  $e (source: $from, target: $to) isa path,
    has strength $s, has traversals $t;
delete $s of $e; delete $t of $e;
insert $e has strength ($s + 1), has traversals ($t + 1);
```

What's stored: The path relation gets its strength incremented and traversal count bumped. This is the pheromone deposit — the trail remembers.

Signal event logged:

```
insert (sender: $from, receiver: $to) isa signal,
  has data "{ \"food\": true }",
  has amount 0.0,
  has success true,
  has ts 2026-04-06T14:32:00;
```

Every signal is an immutable event record. The colony's complete history lives in TypeDB as signal relations.

Turn 4: Harvester Follows Trail — select() with Sensitivity

Runtime — `world.select('food', 0.9)`:

```
// world.ts:215-224 — STAN: Stigmergic A* Navigation
// weight = (1 + pheromone × sensitivity) × latPenalty × revBoost

const viable = Object.entries(strength)
  .filter(([e]) => matchEdge(e, 'food'))
  .map(([e, s]) => {
```

```
    const net = s - (resistance[e] || 0)           // 2 - 0 = 2
    const latPenalty = 1 / (1 + (latency[e] || 0) / 1000)  // 1.0 (no
latency yet)
    const revBoost = 1 + Math.log1p(revenue[e] || 0)       // 1.0 (no
revenue yet)
    return [e, (1 + Math.max(0, 2) * 0.9) * 1.0 * 1.0]    // weight = 2.8
  })

// Probabilistic pick weighted by strength:
// N→SW: weight 2.8 (strong trail)
// N→NW: weight 1.9 (weaker trail)
// Harvester picks SW ~60% of the time
```

Why sensitivity matters:

```
Scout   (0.1): weight = 1 + 2 × 0.1 = 1.2  →  barely prefers SW over NW
Harvester(0.9): weight = 1 + 2 × 0.9 = 2.8  →  strongly prefers SW
```

The same `select()` call, different sensitivity, completely different behavior.
Scouts explore. Harvesters exploit. One formula.

Every Tick: fade() — Asymmetric Decay

Runtime — `world.fade(0.05)`:

```
// world.ts:230-240
for (const e in strength) {
  const age = (Date.now() - (lastUsed[e] || 0)) / 3_600_000  // hours
since last use
  const seasonal = 1 + Math.min(age, 24) / 24                // unused =
faster decay
  strength[e] *= (1 - 0.05 * seasonal)
  const floor = (peak[e] || 0) * 0.05                        // ghost
trails survive
  if (strength[e] < floor) strength[e] = floor
  if (strength[e] < 0.01) delete strength[e]
}
// Resistance decays 2x faster — failures forgive
for (const e in resistance) {
  resistance[e] *= (1 - 0.05 * 2)  // 0.10 decay vs 0.05
  if (resistance[e] < 0.01) delete resistance[e]
}
```

TypeDB — `persist.fade(0.05)`:

```
match $e isa path, has strength $s, has resistance $a;
delete $s of $e; delete $a of $e;
insert $e has strength ($s * 0.95),
       has resistance ($a * 0.90);
```

Three decay mechanics at work:

1. **Asymmetric:** Resistance decays at 2x the rate. A guard's alarm wall on NE costs twice as much "maintenance" as a harvester's food trail. This is why building beats fighting — the math literally favors creation.
2. **Seasonal:** Unused edges decay faster. An abandoned trail to depleted food evaporates at up to 2x speed. Active trails hold steady.
3. **Ghost floor:** Strength never drops below 5% of its peak. Even after the food is gone, a faint memory of the trail persists — a ghost that scouts can rediscover. `peak['N→SW']` remembers the trail's best day.

Turn 12: Guard Alarms NE — warn()

Runtime — `world.warn('NE_corridor', 1)`:

```
// world.ts:125-127
resistance['entry→NE'] = (resistance['entry→NE'] || 0) + 1  // → 1
```

TypeDB — `persist.warn('entry→NE', 1)`:

```
match
  $from isa unit, has uid "entry";
  $to isa unit, has uid "NE";
  $e (source: $from, target: $to) isa path, has resistance $a;
delete $a of $e;
insert $e has resistance ($a + 1);
```

Toxicity check — every signal through this path hits the sandwich:

```
// persist.ts:234-238
const isToxic = (edge: string) => {
  const a = net.resistance[edge] || 0   // resistance
  const s = net.strength[edge] || 0     // strength
  return a >= 10 && a > s * 2 && (a + s) > 5
}
```

At turn 12, NE has resistance 2.3, strength 0. Not toxic yet (needs resistance >= 10). But every guard patrol adds more alarm. By turn 20, if sustained:

- resistance 12, strength 0 → **toxic**. All signals through NE get dissolved before the LLM is ever called. Zero cost. Instant wall.

Turn 14: Imprint SW Highway — know()

Runtime — `persist.know()`:

```
// persist.ts:206-212
const strong = net.highways(100).filter(h => h.strength >= 20)
// N→SW at strength 5.0 — doesn't qualify yet for permanent knowledge
// (needs strength >= 20 in production, but we're playing accelerated)
```

TypeDB — reduce fade rate on proven paths:

```
match $p isa path, has strength $s, has fade-rate $fr;
      $s >= 50.0; $fr > 0.01;
delete $fr of $p;
insert $p has fade-rate 0.01;
```

What "imprint" actually does: It doesn't freeze the trail. It reduces its fade rate to near-zero. The trail still decays — just so slowly it's effectively permanent. This maps to real ant biology: major foraging trails can persist for years through constant reinforcement, but they're not truly immortal.

Hypothesis created:

```
insert $h isa hypothesis,
  has hid "path-N-SW-142",
  has statement "path N→SW is proven (confidence 0.92)",
  has hypothesis-status "confirmed",
  has observations-count 142,
  has p-value 0.01;
```

The colony now **knows** this path works. This knowledge survives server restarts, deploys, and time. It's in the brain, not the nervous system.

The Closed Loop — What Happens Every Tick

```
// loop.ts:53-98 — the growth tick
const tick = async (net, complete) => {
  // L1: SELECT — pick next signal target
  const next = net.select()
```

```
  // L1: Is this a highway? Skip the LLM entirely
  if (netStrength >= 20) {
    // Proven path — no LLM call, no cost, instant routing
    net.mark(edge, chainDepth)
    result.skipped = true
  } else {
    // L1: ASK — signal and wait for outcome
    const outcome = await net.ask({ receiver: next })

    // L2: MARK/WARN — four outcomes, four responses
    if (outcome.result)    net.mark(edge, chainDepth)   // success →
strengthen
    if (outcome.timeout)   /* neutral — not the agent's fault */
    if (outcome.dissolved) net.warn(edge, 0.5)          // missing → mild
warn
    else                   net.warn(edge, 1)            // failure → full
warn
  }

  // L1: DRAIN — process queued signals
  net.drain()

  // L3: FADE — every 5 minutes
  net.fade(0.05)

  // L5: EVOLVE — every 10 minutes (24h cooldown per agent)
  // Query TypeDB for struggling agents, rewrite prompts

  // L6: KNOW — every hour
  // Promote highways to permanent knowledge

  // L7: FRONTIER — every hour
  // Detect unexplored tag clusters
}
```

Chain depth scaling: When signals chain (scout finds food → harvester collects → food returns to nest), each success in the chain marks the path with increasing weight. `chainDepth` caps at 5. A 3-deep chain marks with 3x weight. This is why longer successful chains build highways faster — compound reinforcement.

The Deterministic Sandwich

Every LLM call is wrapped in deterministic checks. This is the security model AND the learning model — they're the same mechanism.

```
PLAYER ACTION
     │
     ▼
PRE-CHECK 1: isToxic(edge)?
  │  resistance >= 10, resistance > 2x strength, samples > 5
  │  YES → dissolve (no LLM call, no cost, instant)
  │  NO  → continue
  ▼
PRE-CHECK 2: capability exists? (TypeDB query)
  │  match (provider: $u, offered: $sk) isa capability;
  │  NO  → dissolve
  │  YES → continue
  ▼
LLM CALL (the one probabilistic step)
  │  generates response
  ▼
POST-CHECK: four outcomes
  │  result    → mark(edge, chainDepth) → trail strengthens
  │  timeout   → neutral (not the agent's fault)
  │  dissolved → warn(edge, 0.5) → mild resistance
  │  nothing   → warn(edge, 1) → full resistance
  ▼
TRAIL UPDATED (both runtime + TypeDB)
```

In game terms: when a scout moves to a hex, the substrate checks if that path is toxic (blocked by alarm pheromone), checks if the scout has the capability to explore, then lets the action happen, then marks or warns based on outcome.

The guard's alarm pheromone literally becomes a firewall — and it's the same `warn()` that handles agent failures. Security and learning are identical.

Hydration — Loading the Colony

When the game starts, the colony loads from TypeDB:

```
// persist.ts:86-102 — load()
const load = async () => {
  // Hydrate all path weights
  const answers = await read(`
    match $e (source: $from, target: $to) isa path,
      has strength $s, has resistance $a;
    $from has uid $fid; $to has uid $tid;
    select $fid, $tid, $s, $a;
  `)
  for (const row of parseAnswers(answers)) {
    net.strength[`${row.fid}→${row.tid}`] = row.s
```

```
    if (row.a > 0) net.resistance[`${row.fid}→${row.tid}`] = row.a
  }

  // Hydrate pending signals (unfinished work)
  const pending = await read(`
    match (sender: $f, receiver: $to) isa signal,
      has success false, has data $d;
    $to has uid $tid;
    select $tid, $d;
  `)
  for (const row of parseAnswers(pending)) {
    net.enqueue({ receiver: row.tid, data: row.d })
  }
}
```

Save game = TypeDB state. Every path, every unit, every signal, every hypothesis. The runtime reconstructs the full colony from the brain.

Close the game at turn 15. Open it tomorrow. `load()` restores:

- All 4 trails with exact strength/resistance values
- The NE alarm wall
- The imprinted SW highway (with reduced fade rate)
- Any pending signals that weren't processed
- Unit stats (success rate, generation, sample count)

The colony remembers everything.

Evolution — When Ants Adapt (L5)

If a scout has < 50% success rate over 20+ samples, the substrate rewrites its prompt:

TypeDB query — find struggling agents:

```
match $u isa unit, has uid $id, has system-prompt $sp,
      has success-rate $sr, has sample-count $sc, has generation $g;
$sr < 0.50; $sc >= 20;
not { $u has last-evolved $le;
      $le > 2026-04-05T14:00:00; };  -- 24h cooldown
(provider: $u, offered: $sk) isa capability;
$sk has skill-id $sid, has tag $tag;
select $id, $sp, $sr, $sc, $g, $sid, $tag;
```

LLM rewrites the prompt. Old prompt saved as hypothesis for rollback:

```
insert $h isa hypothesis,
  has hid "evolve-scout-gen0",
  has statement "gen 0 prompt for scout: You are the Scout...",
  has hypothesis-status "testing",
  has observations-count 0,
  has p-value 1.0;
```

New prompt installed:

```
match $u isa unit, has uid "scout",
      has system-prompt $sp, has generation $g;
delete $sp of $u; delete $g of $u;
insert $u has system-prompt "You are the Scout. Revised...",
       has generation 1,
       has last-evolved 2026-04-06T14:00:00;
```

In game terms: your scouts got smarter. They found food 40% of the time in gen 0. After evolution, gen 1 scouts might find food 65% of the time. The colony learns at two levels — trails (paths) AND agents (prompts).

The Full Transaction Map

Every game action, every TypeDB transaction:

Game Action	Runtime	TypeDB Write	TypeDB Read
Hatch	`add(id)`	`insert unit`	—
Scent	`enqueue(signal)`	`insert signal`	—
Move	`signal(s, from)`	`update path strength`	`preflight(from, to, skill)`
Find food	`mark(edge, 1)`	`update path strength, traversals`	—
Carry food	`mark(edge, depth)`	`update path strength, traversals`	—
Alarm	`warn(edge, 1)`	`update path resistance`	—
Fade	`fade(0.05)`	`update all paths`	—
Toxic check	`isToxic(edge)`	—	`is_safe(from, to)`

Game Action	Runtime	TypeDB Write	TypeDB Read
Select path	`select(type, sens)`	—	`suggest_route(from, skill)`
Follow highway	`follow(type)`	—	`optimal_route(from, skill)`
Imprint	`know()`	`update fade-rate`, `insert hypothesis`	`highways()`
Evolve	LLM rewrite	`update system-prompt`, `insert hypothesis`	`needs_evolution()`
Load game	`load()`	—	`match path`, `match signal`
Save game	`sync()`	`insert all paths`	—

TypeDB Schema — The Colony's Brain

Six dimensions, mapped to game concepts:

```
DIMENSION 1: GROUPS          → Colony teams (nest divisions)
DIMENSION 2: ACTORS (units)  → Ants (scout, harvester, guard, nurse,
queen)
DIMENSION 3: THINGS (skills) → Capabilities (explore, collect, patrol,
tend)
DIMENSION 4: PATHS           → Pheromone trails (strength, resistance,
revenue)
DIMENSION 5: EVENTS (signals)→ Ant actions (every forage, every carry,
every alarm)
DIMENSION 6: KNOWLEDGE       → Colony memory (hypotheses, frontiers,
objectives)
```

Classification functions fire automatically:

```
-- Is this trail a highway? (strength >= 50)
fun is_highway($e: path) -> boolean:
    match $e has strength $s; $s >= 50.0;
    return first true;

-- Is this path safe? (not toxic)
fun is_safe($from: unit, $to: unit) -> boolean:
    match (source: $from, target: $to) isa path,
      has strength $s, has resistance $a;
```

```
    return first if ($a > $s and $a >= 10.0) then false else true;

-- Does this ant need evolution?
fun needs_evolution($u: unit) -> boolean:
    match $u has success-rate $sr, has sample-count $sc;
    $sr < 0.50; $sc >= 20;
    return first true;

-- What status is this trail?
fun path_status($e: path) -> string:
    match $e has strength $s, has resistance $a, has traversals $t;
    return first
        if ($a > $s and $a >= 10.0) then "toxic"
        else if ($s >= 50.0) then "highway"
        else if ($s >= 10.0 and $s < 50.0 and $t < 10) then "fresh"
        else if ($s > 0.0 and $s < 5.0) then "fading"
        else "active";
```

The brain classifies automatically. The runtime just reads the verdict.

Why This Matters

The game isn't a simulation of ants running on a game engine.
The game IS the substrate, visualized.

Every `mark()` in the game is the same `mark()` that production AI agents use.
Every `warn()` is the same firewall. Every `fade()` is the same forgetting.
Every `know()` is the same knowledge crystallization.

You're not playing a game about ant colonies.
You're playing the actual ant colony algorithm that routes AI agents in production.

The hex grid is a skin. The pheromone is real. The TypeDB transactions are real.
The only difference between Colony the game and ONE the platform is the renderer.

```
Colony (game)          ONE (production)
──────────────         ────────────────
hex grid               API endpoints
ant sprites            agent icons
food sources           capabilities
alarm pheromone        toxic path detection
imprint                know()
same engine            same engine
same TypeDB            same TypeDB
same math              same math
```

The game teaches the substrate by being the substrate.

Signal. Mark. Warn. Follow. Fade. Highway. Queue. Evolve. Know.
Nine verbs. One schema. Real transactions. Real persistence. Real emergence.